

PAINTKILLER

Documento de Diseño del Videojuego

Autor: Joel Vas Morales

Proyecto Académico — Unity 2D

Plataforma: PC (Windows)

Multijugador Local Asimétrico (Ratón vs Teclado)

Estado: MVP Académico

0. Que es PaintKiller

PaintKiller es un enfrentamiento directo entre dos personas sentadas frente al mismo ordenador.

Uno crea el problema.

El otro intenta sobrevivir.

No hay historia. No hay lore. No hay narrativa profunda.

Hay tensión corta, presión constante y reacción inmediata.

El Painter dibuja y genera herramientas ofensivas.

El Runner solo se mueve.

Nada más.

El objetivo no es hacer el juego más grande.

Es que funcione. Que se sienta justo. Que genere ese momento de:

“casi te doy”.

Es un proyecto académico, pero está planteado como un sistema serio.

Prefiero algo pequeño y sólido antes que algo grande e inestable.

1. Idea central

La base es la asimetría real.

No solo en habilidades.

También en periférico.

Painter → ratón

Runner → teclado

No juegan al mismo sistema.

El Painter introduce caos.

El Runner lo interpreta y lo gestiona.

Uno genera presión.

El otro la sobrevive.

El objetivo no es que uno domine siempre, sino que ambos sientan control dentro de su rol.

2. Como se juega

Cada ronda dura aproximadamente 60 segundos.

El Runner tiene 3 vidas.

Si pierde las 3 → gana el Painter.

Si el tiempo termina y le queda al menos una vida → gana el Runner.

Las rondas son cortas porque:

- Reducen la frustración.
- Permiten reintento inmediato.
- Mantienen un ritmo alto.
- Favorecen el aprendizaje rápido.

No quiero partidas largas donde uno espere sin jugar.

3. El Painter

El Painter no improvisa durante la partida.

Antes de jugar configura herramientas mediante un editor paramétrico.

Cada herramienta define:

- Forma
- Tamaño
- Duración
- Impacto
- Coste
- Cooldown
- Color

Los perfiles se guardan en JSON.

No es solo una decisión técnica.

Es una forma de separar datos y comportamiento.

Si algo está desbalanceado, se ajusta el perfil.

No se reescribe la lógica.

Esto obliga a pensar el sistema de manera estructurada.

4. El Runner

El Runner no tiene ataque.

Solo movimiento.

Si el movimiento no se siente preciso, el sistema entero pierde coherencia.

Debe ser:

- Inmediato
- Fluido
- Sin retardo perceptible
- Con colisiones claras

El dash se contempla, pero no es obligatorio.
Si el movimiento base es suficiente, no se añade complejidad innecesaria.

Prefiero simplicidad estable antes que mecánicas forzadas.

5. Caos con control

El juego puede generar situaciones absurdas.
Eso es intencionado.

Pero el caos no puede ser injusto.

Para evitar saturación:

- Límite de instancias simultáneas.
- Cooldowns proporcionales.
- Peso estructural por herramienta.
- Hitboxes simplificadas.

Lo visual puede ser expresivo.
Lo que impacta debe ser claro.

Separar colisión y representación no es solo optimización.
Es control del sistema.

6. Arquitectura

El proyecto se divide en capas claras:

Input → captura
Logic → decide
Data → define
Visual → muestra

No existe un manager central que haga todo.

Cada sistema tiene una responsabilidad concreta.

Esto no es exceso de teoría.
Es una decisión para mantener el proyecto entendible y escalable.

7. Alcance real

Incluye:

- Movimiento funcional
- Sistema de rondas
- Herramientas parametrizadas
- Editor independiente
- Persistencia en JSON
- Multijugador local

No incluye:

- Online
- Ranking
- Progresión compleja
- Microtransacciones
- Historia

No por falta de ambición, sino por control de alcance.

Primero hago la base.

Luego expando esta base.

8. Balance y testing

No asumo que el sistema esté equilibrado desde el inicio.

Pruebo.

Ajusto.

Repito.

No busco competencia eSport.

Busco una experiencia divertida y social.

Si uno domina siempre, algo falla.

Si el Runner siente injusticia, algo falla.

Si el Painter no puede presionar, algo falla.

El balance no se teoriza.
Se juega.

9. Riesgos

- Rendimiento por exceso de objetos.
- Desbalance entre roles.
- Scope creep.
- Frustración por diferencias de habilidad.

Mitigación:

- Límites claros.
- Parámetros cerrados.
- Testing continuo.
- MVP bien definido.

Nada crece si la base no es estable.

10. Plan de desarrollo

Fase 1 → Movimiento y estructura base.

Fase 2 → Sistema de herramientas.

Fase 3 → Editor y serialización.

Fase 4 → Balance.

Fase 5 → Pulido.

No se avanza si la fase anterior no es sólida.

11. Competencias que demuestra el proyecto

No pretende demostrar que puedo hacer algo enorme.

Demuestra que:

- Puedo estructurar un sistema.
- Puedo separar datos y lógica.
- Puedo controlar el alcance.
- Puedo priorizar arquitectura sobre expansión.
- Puedo tomar decisiones técnicas con criterio.

12. Cierre

PaintKiller es pequeño a propósito.

Dos jugadores.

Un espacio compartido.

Presión constante.

No busca magnitud.

Busca coherencia.

Si la base funciona, podrá crecer.

Si no, no merece hacerlo.

Primero estructura.

Luego todo lo demás.